

Spatie Laravel Event Sourcing Cheat Sheet

A dense, printable reference for spatie/laravel-event-sourcing: aggregates, projectors, reactors, artisan commands, testing, and the gotchas people hit first.

Printed from <https://albertoarena.it/cheatsheets/spatie-event-sourcing/> by Alberto Arena.

THE FLOW

```
command
  ↓
  aggregate root (guards + recordThat)
    ↓
    persist()
      ↓
      stored_events
        ↓
        projectors build read models / reactors fire side effects
```

BUILDING BLOCKS

Aggregate root: Guards business rules, records events, rebuilt from its event stream. Extends `AggregateRoot`.

Event: An immutable fact that happened. Implements `ShouldBeStored`.

Projector: Builds/updates read models from events. Replayable, side-effect-free.

Reactor: Performs side effects (mail, notifications, HTTP). Not replayed.

Read model / projection: The queryable state projectors maintain.

StoredEvent: The persisted event row in the event store.

Snapshot: Cached aggregate state to avoid replaying long streams.

AGGREGATE ROOT ESSENTIALS

```
AggregateRoot::retrieve($uuid)
```

Rebuild from history.

```
→recordThat(new SomethingHappened(...))
```

Append an event.

```
→persist()
```

Write recorded events to the store.

```
protected function applySomethingHappened(SomethingHappened $event)
```

Mutate in-memory state (no side effects, no queries into other aggregates).

Enforce invariants BEFORE `recordThat`; throw domain exceptions on violation.

Concurrency is built in: `persist()` throws `CouldNotPersistAggregate` if another process persisted events for this aggregate since it was retrieved. No manual version passing needed.

PROJECTORS VS REACTORS (THE RULE PEOPLE GET WRONG)

Projectors: idempotent, replay-safe, only touch their read models.

Side effects (emails, external calls) go in reactors, never projectors, because a replay would re-fire them.

HANDLERS

```
public function onSomethingHappened(SomethingHappened $event)
```

Projector handler method.

Register projectors/reactors (config `event-sourcing.projectors` / at runtime).

Weak-typed catch-all vs typed handler methods.

ARTISAN COMMANDS

```
php artisan make:aggregate
```

Scaffold a new aggregate root.

```
php artisan make:projector
```

Scaffold a new projector. Add `-Q` for a `QueuedProjector`.

```
php artisan make:reactor
```

Scaffold a new reactor.

```
php artisan make:storable-event
```

Scaffold a new domain event.

```
php artisan event-sourcing:replay
```

Replay stored events to projectors.

```
php artisan event-sourcing:list
```

List all registered event handlers.

TESTING

```
AggregateRoot::fake($uuid)→given([ ... ])→when(fn($agg) ⇒ ... )→assertRecorded([new Expected(...)])
```

Also see `→assertNotRecorded(...)`.

GOTCHAS

Projectors must be idempotent and replay-safe.

Don't query other aggregates inside `apply*`.

Version events; upcast when the shape changes.

Snapshot long-lived aggregates.

WHEN NOT TO USE EVENT SOURCING

Simple CRUD, no audit/temporal requirement, no complex invariants: the cost isn't worth it. Use it when history, audit, or rich domain rules matter.